

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

**Дисциплина:
«Программирование»**

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №5
«Объектно-ориентированное программирование в Python»
Вариант: 1-3

Выполнил:

студент гр. N3151

Агакарян Арам Арманович



(подпись)

Проверил:

инженер ФБИТ

Дронов Вадим Юрьевич

(отметка о выполнении)

(подпись)

Санкт-Петербург

2026 г.

ВВЕДЕНИЕ

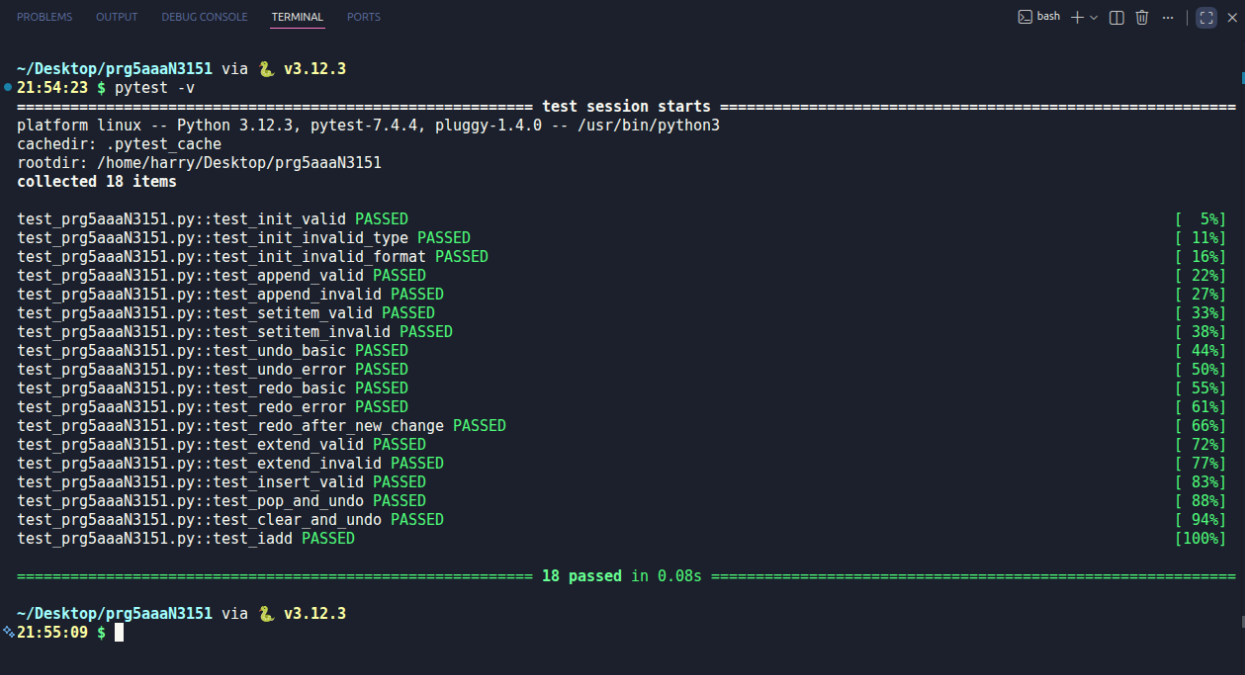
Цель работы – Разработать на языке Python для ОС Linux модуль, содержащий определение класса, наследующего от list, который хранит только MAC-адреса в формате XX:XX:XX:XX:XX:XX и поддерживает историю изменений с методами undo() и redo(). Также разработать набор тестов для системы pytest для проверки корректности реализации

Задание

Вариант - 1-3 – list - MAC-адрес

Примеры работы программ

Примеры работы программ на различных исходных данных (скриншоты). Ссылаться на них как на рисунки.



```
~ /Desktop/prg5aaaN3151 via v3.12.3
21:54:23 $ pytest -v
===== test session starts =====
platform linux -- Python 3.12.3, pytest-7.4.4, pluggy-1.4.0 -- /usr/bin/python3
cachedir: .pytest cache
rootdir: /home/harry/Desktop/prg5aaaN3151
collected 18 items

test_prg5aaaN3151.py::test_init_valid PASSED [ 5%]
test_prg5aaaN3151.py::test_init_invalid_type PASSED [ 11%]
test_prg5aaaN3151.py::test_init_invalid_format PASSED [ 16%]
test_prg5aaaN3151.py::test_append_valid PASSED [ 22%]
test_prg5aaaN3151.py::test_append_invalid PASSED [ 27%]
test_prg5aaaN3151.py::test_setitem_valid PASSED [ 33%]
test_prg5aaaN3151.py::test_setitem_invalid PASSED [ 38%]
test_prg5aaaN3151.py::test_undo_basic PASSED [ 44%]
test_prg5aaaN3151.py::test_undo_error PASSED [ 50%]
test_prg5aaaN3151.py::test_redo_basic PASSED [ 55%]
test_prg5aaaN3151.py::test_redo_error PASSED [ 61%]
test_prg5aaaN3151.py::test_redo_after_new_change PASSED [ 66%]
test_prg5aaaN3151.py::test_extend_valid PASSED [ 72%]
test_prg5aaaN3151.py::test_extend_invalid PASSED [ 77%]
test_prg5aaaN3151.py::test_insert_valid PASSED [ 83%]
test_prg5aaaN3151.py::test_pop_and_undo PASSED [ 88%]
test_prg5aaaN3151.py::test_clear_and_undo PASSED [ 94%]
test_prg5aaaN3151.py::test_iadd PASSED [100%]

===== 18 passed in 0.08s =====

~ /Desktop/prg5aaaN3151 via v3.12.3
21:55:09 $
```

1 ИСХОДНЫЙ КОД ПРОГРАММЫ

Код (с комментариями!). В данном случае разрешается вставить большой код в виде листинга сюда.

```
import re

from collections import deque

class FormatError(Exception):
    pass

class UndoError(Exception):
    pass

class RedoError(Exception):
    pass

MAC_PATTERN = re.compile(r'^([0-9a-fA-F]{2}:){5}[0-9a-fA-F]{2}$')

def validate_mac(value):
    if not isinstance(value, str):
        raise TypeError(f'Expected str, got {type(value).__name__}')
    if not MAC_PATTERN.match(value):
        raise FormatError(f'Invalid MAC address format: {value}')
    return value

class MyMACList(list):
```

```

def __init__(self, iterable=None):
    self._history = deque()
    self._redo_stack = deque()
    self._record_state()

    if iterable is None:
        super().__init__()
    else:
        validated = [validate_mac(item) for item in iterable]
        super().__init__(validated)
        self._record_state()

    def _record_state(self):
        self._history.append(list(self))
        self._redo_stack.clear()

    def _check_and_record(self, operation):
        result = operation()
        self._record_state()
        return result

    def append(self, item):
        validated = validate_mac(item)
        super().append(validated)
        self._record_state()

    def insert(self, index, item):

```

```

validated = validate_mac(item)
super().insert(index, validated)
self._record_state()

def extend(self, iterable):
    validated = [validate_mac(i) for i in iterable]
    super().extend(validated)
    self._record_state()

def __setitem__(self, key, value):
    if isinstance(key, slice):
        validated = [validate_mac(v) for v in value]
        super().__setitem__(key, validated)
    else:
        super().__setitem__(key, validate_mac(value))
    self._record_state()

def __delitem__(self, key):
    super().__delitem__(key)
    self._record_state()

def pop(self, index=-1):
    result = super().pop(index)
    self._record_state()
    return result

def remove(self, value):
    idx = self.index(validate_mac(value))

```

```

super().pop(idx)
self._record_state()

def clear(self):
    super().clear()
    self._record_state()

def __iadd__(self, other):
    validated = [validate_mac(item) for item in other]
    super().__iadd__(validated)
    self._record_state()
    return self

def undo(self):
    if len(self._history) <= 1:
        raise UndoError("No more undo steps available")
    self._redo_stack.append(list(self))
    self._history.pop()
    prev_state = self._history[-1]
    super().clear()
    super().extend(prev_state)

def redo(self):
    if not self._redo_stack:
        raise RedoError("No more redo steps available")
    next_state = self._redo_stack.pop()
    self._history.append(next_state)

```

```
super().clear()
super().extend(next_state)
```

test.py

```
import pytest
from prg5aaaN3151 import MyMACList, FormatError, UndoError,
RedoError


def test_init_valid():
    lst = MyMACList(["aa:bb:cc:dd:ee:ff", "00:11:22:33:44:55"])
    assert lst == ["aa:bb:cc:dd:ee:ff", "00:11:22:33:44:55"]


def test_init_invalid_type():
    with pytest.raises(TypeError):
        MyMACList([123, "aa:bb:cc:dd:ee:ff"])


def test_init_invalid_format():
    with pytest.raises(FormatError):
        MyMACList(["invalid", "aa:bb:cc:dd:ee:ff"])


def test_append_valid():
    lst = MyMACList()
    lst.append("ab:cd:ef:01:23:45")
    assert lst == ["ab:cd:ef:01:23:45"]


def test_append_invalid():
    lst = MyMACList()
    with pytest.raises(FormatError):
```

```

lst.append("192.168.0.1")

def test_setitem_valid():
    lst = MyMACList(["aa:aa:aa:aa:aa:aa"])
    lst[0] = "bb:bb:bb:bb:bb:bb"
    assert lst[0] == "bb:bb:bb:bb:bb:bb"

def test_setitem_invalid():
    lst = MyMACList(["aa:aa:aa:aa:aa:aa"])
    with pytest.raises(FormatError):
        lst[0] = "not a mac"

def test_undo_basic():
    lst = MyMACList(["a1:b2:c3:d4:e5:f6"])
    lst.append("11:22:33:44:55:66")
    assert lst == ["a1:b2:c3:d4:e5:f6", "11:22:33:44:55:66"]
    lst.undo()
    assert lst == ["a1:b2:c3:d4:e5:f6"]

def test_undo_error():
    lst = MyMACList()
    with pytest.raises(UndoError):
        lst.undo()

def test_redo_basic():
    lst = MyMACList(["aa:bb:cc:dd:ee:ff"])
    lst.append("11:22:33:44:55:66")
    lst.undo()

```

```

assert lst == ["aa:bb:cc:dd:ee:ff"]
lst.redo()
assert lst == ["aa:bb:cc:dd:ee:ff", "11:22:33:44:55:66"]

def test_redo_error():
    lst = MyMACList(["00:00:00:00:00:00"])
    with pytest.raises(RedoError):
        lst.redo()

def test_redo_after_new_change():
    lst = MyMACList(["11:11:11:11:11:11"])
    lst.append("22:22:22:22:22:22")
    lst.undo()
    lst.append("33:33:33:33:33:33")
    with pytest.raises(RedoError):
        lst.redo()

def test_extend_valid():
    lst = MyMACList(["aa:aa:aa:aa:aa:aa"])
    lst.extend(["bb:bb:bb:bb:bb:bb", "cc:cc:cc:cc:cc:cc"])
    assert lst == ["aa:aa:aa:aa:aa:aa", "bb:bb:bb:bb:bb:bb", "cc:cc:cc:cc:cc:cc"]

def test_extend_invalid():
    lst = MyMACList()
    with pytest.raises(FormatError):
        lst.extend(["bad", "aa:aa:aa:aa:aa:aa"])

def test_insert_valid():

```

```

lst = MyMACList(["aa:aa:aa:aa:aa:aa", "cc:cc:cc:cc:cc:cc"])
lst.insert(1, "bb:bb:bb:bb:bb:bb")
assert lst[1] == "bb:bb:bb:bb:bb:bb"

def test_pop_and_undo():
    lst = MyMACList(["aa:aa:aa:aa:aa:aa", "bb:bb:bb:bb:bb:bb"])
    lst.pop()
    assert lst == ["aa:aa:aa:aa:aa:aa"]
    lst.undo()
    assert lst == ["aa:aa:aa:aa:aa:aa", "bb:bb:bb:bb:bb:bb"]

def test_clear_and_undo():
    lst = MyMACList(["aa:aa:aa:aa:aa:aa", "bb:bb:bb:bb:bb:bb"])
    lst.clear()
    assert lst == []
    lst.undo()
    assert lst == ["aa:aa:aa:aa:aa:aa", "bb:bb:bb:bb:bb:bb"]

def test_iadd():
    lst = MyMACList(["aa:aa:aa:aa:aa:aa"])
    lst += ["bb:bb:bb:bb:bb:bb"]
    assert lst == ["aa:aa:aa:aa:aa:aa", "bb:bb:bb:bb:bb:bb"]
    lst.undo()
    assert lst == ["aa:aa:aa:aa:aa:aa"]

```

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы №5 был разработан модуль на языке Python, содержащий класс MyMACList, наследующий от встроенного типа list, который хранит только строки в формате MAC-адреса (XX:XX:XX:XX:XX:XX). Реализован механизм истории изменений с методами `undo()` и `redo()`, позволяющий откатывать и повторять операции, изменяющие содержимое списка. Созданы собственные исключения `FormatError`, `UndoError`, `RedoError` для обработки ошибочных ситуаций. Разработан набор из 18 тестов для системы `pytest`, покрывающих все переопределённые и унаследованные методы, а также функциональность `undo()` и `redo()`.

Все тесты успешно пройдены. Программа соответствует требованиям задания и работает в ОС Linux с Python 3.12.3.

